MLFQ Scheduler Implementation - Final Report

Project: Multi-Level Feedback Queue Scheduler for xv6-RISC-V
Team Members: Saad Inam - 29068, Hassan Jabbar - 29060

Executive Summary

This project successfully implements a Multi-Level Feedback Queue (MLFQ) scheduler in xv6-RISC-V, replacing the default round-robin scheduler. The implementation includes:

- ✅ 4-level priority queue system
- ✅ Dynamic process priority adjustment based on CPU usage
- ✅ Starvation prevention through periodic boosting
- ✅ `getprocinfo()` system call for monitoring
- ✅ Comprehensive testing with CPU-bound and I/O-bound processes

Result: A functioning priority-based scheduler that provides better responsiveness for interactive processes while maintaining fairness for CPU-intensive tasks.

---

1. Implementation Details

1.1 Core Components

Modified Files:

1. **kernel/proc.h** - Added MLFQ fields to `struct proc` and `struct`

`procinfo`

2. **kernel/proc.c** - Implemented MLFQ scheduling logic
3. **kernel/param.h** - Defined MLFQ constants
4. **kernel/trap.c** - Added tick counting and boosting
5. **kernel/defs.h** - Exported `mlfq_boost()` function
6. **kernel/sysproc.c** - Implemented `getprocinfo()` syscall
7. **kernel/syscall.h/c** - Registered syscall
8. **user/user.h** - User-space syscall declaration
9. **user/usys.pl** - Syscall stub generation

New Files:

1. **user/mlfqtest.c** - Test program demonstrating MLFQ behavior
2. **DESIGN_DOCUMENT.md** - Design specification
3. **FINAL_REPORT.md** - This document

1.2 Data Structures

```
// Per-process MLFQ fields
struct proc {
  int queue_level;        // Current queue (0-3)
  int ticks_used;         // Ticks in current queue
  struct proc *next_proc; // Queue linked list
  int quantum;            // Time slice for current level
};

// Global queue structure
struct {
  struct proc *head;
  struct proc *tail;
} mlfq[4];  // 4 priority levels

// Boost counter
static uint boost_counter = 0;
```

1.3 Key Algorithms

Scheduling (scheduler function):

```
void scheduler(void) {
  for(;;) {
    p = mlfq_next();  // Get highest priority runnable process
    if(p != 0) {
      p->state = RUNNING;
      swtch(&c->context, &p->context);
      release(&p->lock);
    } else {
      wfi();  // Wait for interrupt
    }
  }
}
```

Demotion (yield function):

```
void yield(void) {
  if(p->ticks_used >= p->quantum) {
    mlfq_demote(p);  // Move to lower queue
  }
  p->state = RUNNABLE;
  mlfq_enqueue(p, p->queue_level);
  sched();
}
```

Boosting (timer interrupt):

```
if(which_dev == 2) {  // Timer interrupt
  p->ticks_used++;
  boost_counter++;

  if(boost_counter >= BOOST_INTERVAL) {
    boost_counter = 0;
    mlfq_boost();  // Reset all to queue 0
  }
```

```
  yield();
}
```

---

2. Testing Results

2.1 Test Setup

- **Platform:** xv6-RISC-V on QEMU
- **Configuration:** Single CPU (CPUS := 1)
- **Test Programs:**
- `mlfqtest` - Mixed CPU/IO workload

2.2 Test Case 1: CPU-bound Process

Expected Behavior: Process should demote from queue 0 → 1 → 2 → 3

Sample Output:

```
CPU bound process [PID 3] starting
CPU [PID 3]: queue=0, ticks=3, quantum=4
CPU [PID 3]: queue=1, ticks=1, quantum=8
CPU [PID 3]: queue=1, ticks=3, quantum=8
CPU [PID 3]: queue=1, ticks=6, quantum=8
CPU [PID 3]: queue=2, ticks=0, quantum=16
CPU [PID 3]: queue=3, ticks=32, quantum=32
CPU [PID 3]: queue=3, ticks=32, quantum=32
CPU [PID 3]: queue=0, ticks=0, quantum=4  // Boosted
```

Result: ✅ PASS - Process correctly demotes and gets boosted

2.3 Test Case 2: I/O-bound Process

Expected Behavior: Process should remain at high priority (queue 0)

Sample Output:

```
I/O bound process [PID 4] starting
I/O [PID 4]: queue=0, ticks=0, quantum=4
I/O [PID 4]: queue=0, ticks=0, quantum=4
I/O [PID 4]: queue=0, ticks=0, quantum=4
I/O [PID 4]: queue=0, ticks=0, quantum=4
```

Result: ✅ PASS - Process stays at queue 0 (never uses full quantum)

2.4 Test Case 3: Mixed Workload

Expected Behavior:

- CPU-bound process demotes to lower queues
- I/O-bound process maintains high priority
- Both eventually complete

Observations:

- I/O process completes quickly with minimal waiting
- CPU process makes steady progress despite demotion
- Boosting prevents starvation

Result: ✅ PASS - Correct priority differentiation

2.5 Test Case 4: getprocinfo() Syscall

Test: `c struct procinfo info; getprocinfo(getpid(), &info); printf("PID=%d, queue=%d, ticks=%d\n", info.pid, info.queue_level, info.ticks_used);`

Result: ✅ PASS - Syscall returns correct information

---

3. Challenges Encountered

3.1 Lock Management

Challenge: Risk of deadlock when manipulating queues and acquiring process locks

Solution:

- `mlfq_next()` acquires process lock before removing from queue
- Consistent lock ordering: always acquire `p->lock` after checking state
- `mlfq_boost()` carefully handles RUNNING processes (not in queues)

3.2 Running Process State

Challenge: Running process not in any queue, but needs boosting

Solution: `c if(p->state == RUNNING) { // Update fields without enqueuing p->queue_level = 0; p->quantum = get_quantum(0); p->ticks_used = 0; }`

3.3 I/O-bound Process Priority

Challenge: I/O processes shouldn't be demoted for blocking

Solution:

- Only demote in `yield()` when quantum is used
- `wakeup()` re-queues at current level (no demotion)
- Preserves high priority for interactive processes

---

4. Testing Methodology

4.1 Unit Testing Approach

1. **Queue operations** - Tested enqueue/dequeue in isolation
2. **Demotion logic** - Verified quantum enforcement
3. **Boosting** - Confirmed all processes reset to level 0
4. **Syscall** - Validated correct data returned

4.2 Integration Testing

1. **Single process** - Basic scheduler functionality
2. **Multiple processes** - Queue priority ordering
3. **Mixed workloads** - CPU vs I/O differentiation
4. **Long-running** - Starvation prevention

4.3 Debug Tools Used

- `printf()` statements in kernel
- `getprocinfo()` for runtime monitoring

How to Build and Test

Building:

```
make clean
make qemu
```

Running Tests:

```
# In xv6 shell:
$ mlfqtest
```

End of Report